Machine Learning - TP1

Système expert : application à la résolution des tours de Hanoï

Vincent Tourenne



Table des Matières

I - Introduction 1
II - Réalisation 2
III - Résultats et Analyse des Résultats 6

I - Introduction

L'objectif de ce TP est de mettre en place un **système à base de règles** (expert) afin de résoudre le problème des **tours de Hanoï**.

Ce jeu de réflexion a pour but de **déplacer** des **palets de taille croissante** d'un **pic de départ** à un **pic d'arrivée** à l'aide d'un pic intermédiaire, selon certaines **contraintes** (ne déplacer qu'un palet à la fois, ne poser un palet que sur un autre de taille supérieure ou un emplacement vide). Dans ce projet, nous ne considérons dans un premier temps que le cas classique : **trois tours** et **trois palets**.

La **base experte** nous fournit les **règles** suivantes pour notre système :

- Un déplacement de palet d'un pic A vers un pic B est considéré comme possible si :
    - Le pic A n'est pas vide ;
    - Le pic B est vide, ou son palet supérieur est plus grand que le palet déplacé ;
    - Le déplacement ne mène pas à une situation déjà observée ;

- Quatre règles définissent quel déplacement effectuer à chaque tour (en considérant les pics A, B et C, A étant le pic de départ et C le pic d'arrivée). En cas de conflit, appliquer la règle au plus petit indice :
    1. Déplacer le dernier palet du pic A vers le pic B si possible ;
    2. Déplacer le dernier palet du pic B vers le pic C si possible ;
    3. Déplacer le dernier palet du pic C vers le pic B si possible ;
    4. Déplacer le dernier palet du pic B vers le pic A si possible.

II - Réalisation

On commence par représenter, en Python, une classe ainsi que l'état initial d'un jeu de Hanoï.

```
class Jeu_Hanoi:
    def __init__(self):
        self.pic = np.zeros([3,3], dtype= int)
        self.nombre_palet = np.zeros(3, dtype=int)
        pass

# Situation initiale :
jeu = Jeu_Hanoi()
jeu.nombre_palet[0] = 3
jeu.pic[0,0] = 3
jeu.pic[0,1] = 2
jeu.pic[0,2] = 1
```

Figure 1 - Extrait de code : classe et état initial d'un jeu de Hanoï

On peut ainsi représenter différents états du jeu à l'aide d'un simple appel `print(jeu.pic)` :

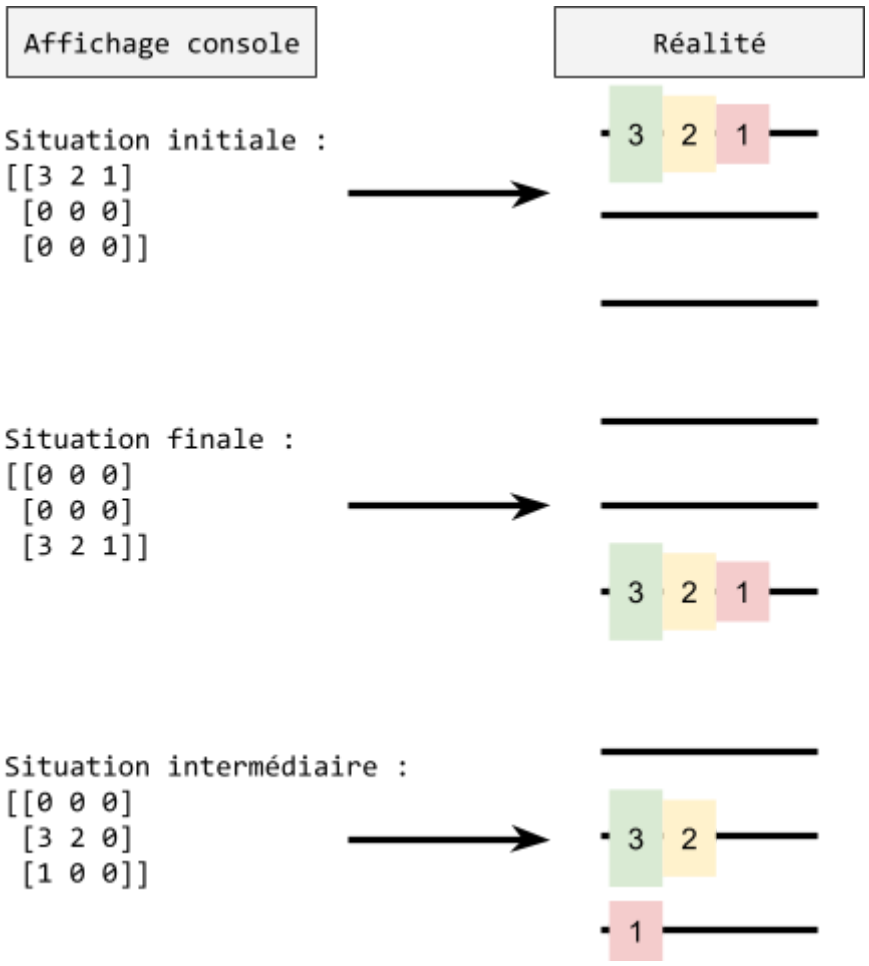


Figure 2 - Représentation d'un état du jeu de Hanoï

Pour notre système expert, il nous faut alors développer un moteur d'inférence en chaînage avant afin de passer de l'état initial à la situation finale. Celui-ci nécessite tout d'abord d'être capable **d'observer** une **situation de jeu**, afin de ne pas la reproduire. On code donc une méthode traduisant une situation de jeu en clé, ajoutée à une liste de situations observées :

```
def nombre_situation(jeu):
    chaine = ""
    for pile in range(2, -1, -1):
        for palet in (3,2,1):
            chaine += "1" if palet in jeu.pic[pile] else "0"
    return int(chaine, 2)

situation_etudiee = [nombre_situation(jeu)]
```

Figure 3 - Extrait de code : encodage binaire d'une situation de jeu

Cet encodage d'une situation de jeu dans un nombre binaire nous permet aisément de stocker et comparer plusieurs états.

Ainsi, lorsqu'on veut **déterminer** si un coup mène à une situation **inédite**, on peut le simuler sur une copie du jeu actuel et observer si le résultat correspond à une situation précédente :

```
def situation_non_vue(pic1, pic2, jeu, situation_etudiee):
    # simuler le déplacement sur une copie temporaire
    jeu_temp = Jeu_Hanoi()
    jeu_temp.pic = np.copy(jeu.pic)
    jeu_temp.nombre_palet = np.copy(jeu.nombre_palet)

    effectuer_deplacement(pic1, pic2, jeu_temp)

    # vérifier si la situation résultante a déjà été vue
    situation = nombre_situation(jeu_temp)
    return situation not in situation_etudiee
```

Figure 4 - Extrait de code : détection d'une situation déjà observée

Nous disposons alors d'une méthode permettant de prédire si un coup reviendra à un état existant ou non.

Ensuite, on a besoin de savoir si un déplacement est **possible** selon les **règles du jeu** de Hanoï, indépendamment de s'il a déjà été vu. Ainsi, on vérifie lors d'un déplacement :

- Si le pic de **départ** n'est pas **vide** ;
- Si le palet le plus haut du pic de destination est **plus grand** que celui déplacé, ou si le pic de **destination** est **vide**.

```python
def pic_vide(indice_pic, jeu):
    check_nb_palet = jeu.nombre_palet[indice_pic] == 0
    check_valeurs_pic = jeu.pic[indice_pic, 0] == 0
    return check_nb_palet or check_valeurs_pic

def regle_jeu(pic1, pic2, jeu): # On déplace de pic1 vers pic2
    if pic_vide(pic1, jeu):
        return False
    else:
        palet1 = jeu.pic[pic1, jeu.nombre_palet[pic1] - 1]
        palet2 = jeu.pic[pic2, jeu.nombre_palet[pic2] - 1]
        plus_petit = palet1 < palet2
        return plus_petit or pic_vide(pic2, jeu)
```

Figure 5 - Extrait de code : vérification de la validité d'un déplacement

Enfin, la dernière méthode nécessaire à notre moteur d'inférence est le **déplacement de palet** en lui-même :

```python
def effectuer_deplacement(pic1, pic2, jeu):
    if regle_jeu(pic1, pic2, jeu):
        # obtenir le palet à déplacer
        palet = jeu.pic[pic1, jeu.nombre_palet[pic1] - 1]

        # déplacer le palet
        jeu.pic[pic2, jeu.nombre_palet[pic2]] = palet
        jeu.pic[pic1, jeu.nombre_palet[pic1] - 1] = 0

        # mettre à jour les compteurs de palets
        jeu.nombre_palet[pic1] -= 1
        jeu.nombre_palet[pic2] += 1
    pass
```

Figure 6 - Extrait de code : déplacement d'un palet

Une fois le déplacement validé, le palet supérieur du pic d'origine est copié à la première position du pic de destination, puis retiré de son emplacement initial. Les valeurs de `nombre_palet` des deux pics sont enfin mises à jour afin que la représentation du jeu reste cohérente avec le contenu du tableau `pic`.

Toutes nos méthodes maintenant disponibles, il ne nous reste plus qu'à écrire la **boucle** de notre moteur d'inférence appliquant les règles de la base experte. Celle-ci se terminera lorsque la situation de notre jeu correspond à celle de son état fini, lorsqu'on atteint un nombre aberrant de coups, ou lorsqu'aucun déplacement admissible et non visité n'est possible.

```
while cmpt < 1000 and nombre_situation(jeu) != nombre_situation(jeu_final):
    pic1, pic2 = 0, 0
    # selectionne une regle eligible
    if regle_jeu(0, 1, jeu) and situation_non_vue(0, 1, jeu, situation_etudiee):
        pic1 = 0
        pic2 = 1
    elif regle_jeu(1, 2, jeu) and situation_non_vue(1, 2, jeu, situation_etudiee):
        pic1 = 1
        pic2 = 2
    elif regle_jeu(2, 1, jeu) and situation_non_vue(2, 1, jeu, situation_etudiee):
        pic1 = 2
        pic2 = 1
    elif regle_jeu(1, 0, jeu) and situation_non_vue(1, 0, jeu, situation_etudiee):
        pic1 = 1
        pic2 = 0
    else:
        print("Aucun déplacement possible ou toutes les situations ont été vues.")
        break

    # effectue le deplacement
    effectuer_deplacement(pic1, pic2, jeu)

    # met à jour la liste des situations
    situation_etudiee.append(nombre_situation(jeu))

    # incrémente le compteur
    cmpt += 1

    # affiche l'etape
    print(f"Étape {cmpt}:")
    print(jeu.pic)
```

Figure 7 - Extrait de code : moteur d'inférence

L'ordre de **priorité des règles** de déplacement établies par la base experte est simulé par l'ordre de **test de possibilités** : la première règle applicable (légale et menant à une situation non vue) rencontrée est toujours celle retenue pour effectuer un déplacement. La nouvelle configuration est ensuite ajoutée aux situations étudiées avant de poursuivre l'inférence. Le processus s'arrête quand la situation finale est atteinte, ou lorsqu'aucune règle ne peut plus être appliquée.

III - Résultats et Analyse des Résultats

On obtient, au déroulement du moteur d'inférence en chaînage avant de ce système expert, un total de **vingt-six étapes** pour résoudre le jeu :

```
Situation initiale :
[[3 2 1]
 [0 0 0]
 [0 0 0]]
Étape 1:
[[3 2 0]
 [1 0 0]
 [0 0 0]]
Étape 2:
[[3 2 0]
 [0 0 0]
 [1 0 0]]
Étape 3:
[[3 0 0]
 [2 0 0]
 [1 0 0]]
Étape 4:
[[3 0 0]
 [2 1 0]
 [0 0 0]]
Étape 5:
[[3 1 0]
 [2 0 0]
 [0 0 0]]
Étape 6:
[[3 1 0]
 [0 0 0]
 [2 0 0]]
Étape 7:
[[3 0 0]
 [1 0 0]
 [2 0 0]]
Étape 8:
[[3 0 0]
 [0 0 0]
 [2 1 0]]
Étape 9:
[[0 0 0]
 [3 0 0]
 [2 1 0]]
Étape 10:
[[0 0 0]
 [3 1 0]
 [2 0 0]]
Étape 11:
[[1 0 0]
 [3 0 0]
 [2 0 0]]
Étape 12:
[[1 0 0]
 [3 2 0]
 [0 0 0]]
Étape 13:
[[0 0 0]
 [3 2 1]
 [0 0 0]]
Étape 14:
[[0 0 0]
 [3 2 0]
 [1 0 0]]
Étape 15:
[[2 0 0]
 [3 0 0]
 [1 0 0]]
Étape 16:
[[2 0 0]
 [3 1 0]
 [0 0 0]]
Étape 17:
[[2 1 0]
 [3 0 0]
 [0 0 0]]
Étape 18:
[[2 1 0]
 [0 0 0]
 [3 0 0]]
Étape 19:
[[2 0 0]
 [1 0 0]
 [3 0 0]]
Étape 20:
[[2 0 0]
 [0 0 0]
 [3 1 0]]
Étape 21:
[[0 0 0]
 [2 0 0]
 [3 1 0]]
Étape 22:
[[0 0 0]
 [2 1 0]
 [3 0 0]]
Étape 23:
[[1 0 0]
 [2 0 0]
 [3 0 0]]
Étape 24:
[[1 0 0]
 [0 0 0]
 [3 2 0]]
Étape 25:
[[0 0 0]
 [1 0 0]
 [3 2 0]]
Étape 26:
[[0 0 0]
 [0 0 0]
 [3 2 1]]
```

Figure 8 - Séquence des situations obtenues par le moteur d'inférence initial

La résolution optimale d'un tel jeu prend seulement **sept coups**. On en est donc loin pour le moment.

Afin de s'approcher de la solution parfaite, on peut réfléchir à l'ajout de règles supplémentaires.

On considère un jeu aux pics A, B et C où A est le pic de départ et C le pic de destination. On peut naïvement se dire que le but du jeu est toujours de déplacer le disque le plus grand disponible des pics A et B vers le pic C. Plus précisément, en observant les résultats du système expert, une optimisation possible porte sur les étapes 4 et 5 :

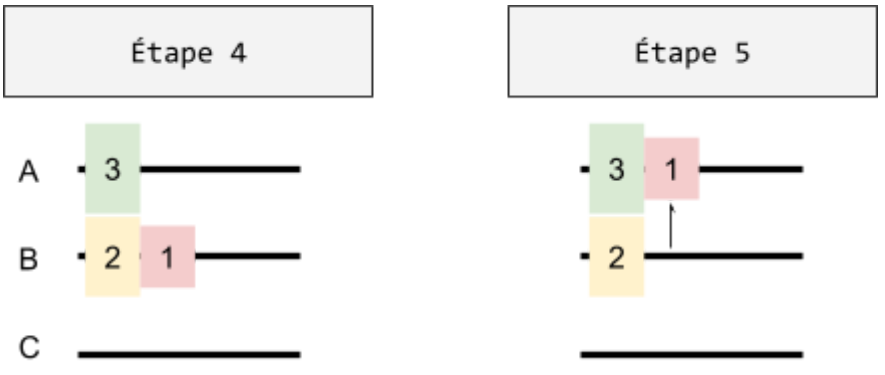


Figure 9 - Déplacement sous-optimal entre les étapes 4 et 5

Selon les règles actuelles, le palet de taille 1 est déplacé sur le pic A.

Pourtant, le palet de plus grande taille de tout le jeu était disponible sur le pic A. Le pic C était vide, donc un **déplacement A vers C** aurait été nettement plus intéressant. Un tel déplacement n'est cependant pas couvert par les règles actuelles fournies par la base experte. Ainsi, une première piste naïve d'optimisation pourrait être de l'ajouter, comme règle la plus prioritaire.

Une fois cet ajustement effectué, un nouveau tour de notre moteur d'inférence donne :

```
Situation initiale :      Étape 5:
[[3 2 1]                  [[0 0 0]
 [0 0 0]                   [2 0 0]
 [0 0 0]]                  [3 1 0]]
Étape 1:                  Étape 6:
[[3 2 0]                  [[2 0 0]
 [0 0 0]                   [0 0 0]
 [1 0 0]]                  [3 1 0]]
Étape 2:                  Étape 7:
[[3 0 0]                  [[2 0 0]
 [2 0 0]                   [1 0 0]
 [1 0 0]]                  [3 0 0]]
Étape 3:                  Étape 8:
[[3 0 0]                  [[0 0 0]
 [2 1 0]                   [1 0 0]
 [0 0 0]]                  [3 2 0]]
Étape 4:                  Étape 9:
[[0 0 0]                  [[0 0 0]
 [2 1 0]                   [0 0 0]
 [3 0 0]]                  [3 2 1]]
```

Figure 10 - Séquence des situations obtenues après l'ajout de la règle 0 : A → C

En introduisant cette simple règle, on a ainsi abaissé le nombre de coups de notre solution de **vingt-six** à seulement **neuf**.

On peut cependant toujours optimiser. En observant les étapes 4 et 5, on constate à nouveau un déplacement contre-productif à notre situation optimale :

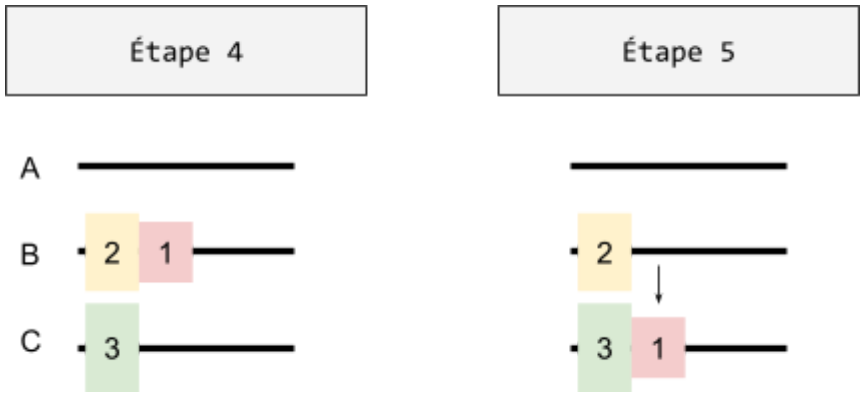


Figure 11 - Nouveau déplacement sous-optimal entre les étapes 4 et 5

Comme il est possible de déplacer un palet du pic B au pic C, c'est la règle qui est choisie par le moteur d'inférence. Or, ce déplacement bloque directement le pic C, sur lequel on préfèrerait installer le palet 2. Il faut alors à l'IA symbolique plusieurs coups supplémentaires pour dégager le pic C et résoudre la situation.

Il serait possible de rajouter des règles de déplacement dans l'ordre des priorités afin de couvrir cette situation spécifique, créant une solution spécialisée à notre problème. On peut cependant chercher une résolution plus généraliste, en **interdisant les déplacements ne prolongeant pas la solution**, c'est-à-dire déplaçant un palet dont la taille n'est pas directement inférieure au premier palet du pic C (dans le cas où celui-ci n'est pas vide, auquel cas le déplacement est autorisé).

Cette dernière règle nous permet alors d'obtenir la solution :

```
Situation initiale :
[[3 2 1]
 [0 0 0]
 [0 0 0]]
Étape 1:
[[3 2 0]
 [0 0 0]
 [1 0 0]]
Étape 2:
[[3 0 0]
 [2 0 0]
 [1 0 0]]
Étape 3:
[[3 0 0]
 [2 1 0]
 [0 0 0]]
Étape 4:
[[0 0 0]
 [2 1 0]
 [3 0 0]]
Étape 5:
[[1 0 0]
 [2 0 0]
 [3 0 0]]
Étape 6:
[[1 0 0]
 [0 0 0]
 [3 2 0]]
Étape 7:
[[0 0 0]
 [0 0 0]
 [3 2 1]]
```

Figure 12 - Séquence des situations obtenues après l'ajout de la règle

On constate ainsi une résolution en **sept coups**, identique à la **solution optimale** du jeu de Hanoi avec trois pics et trois palets.